MACROVA

A PERSONALIZED DIET PLANNING AND NUTRITION TRACKING WEB APPLICATION

A PROJECT REPORT SUBMITTED TO

[NAME OF COLLEGE / UNIVERSITY]

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

BY

[STUDENT NAME] | [REGISTER NUMBER]

UNDER THE GUIDANCE OF

[PROJECT GUIDE NAME AND DESIGNATION]

DEPARTMENT OF COMPUTER APPLICATIONS

[COLLEGE / UNIVERSITY NAME]

ACADEMIC YEAR: [YEAR]

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “MACROVA: A PERSONALIZED DIET PLANNING AND NUTRITION TRACKING WEB APPLICATION” is a bona fide record of the project work carried out by [STUDENT NAME] (Register No. [REGISTER NUMBER]) under my supervision and guidance, in partial fulfilment of the requirements for the award of the Bachelor of Computer Applications degree.

To the best of my knowledge, the work presented in this report is the candidate’s project work. The final title, institutional details, candidate particulars, dates, signatures, and approval must be reviewed and completed by the candidate and the college before submission.

DECLARATION

I, [STUDENT NAME] (Register No. [REGISTER NUMBER]), hereby declare that the project report entitled “MACROVA: A PERSONALIZED DIET PLANNING AND NUTRITION TRACKING WEB APPLICATION” submitted to [COLLEGE / UNIVERSITY NAME] in partial fulfilment of the requirements for the Bachelor of Computer Applications degree is a record of project work undertaken by me.

I further declare that this report has been prepared using the Macrova application source code available in the project repository and that it has not been submitted, in whole or in part, to another institution for any degree or qualification. Any external material used is acknowledged in the references. The declaration and originality statement should be checked against the institution’s prescribed wording before signing.

Place: [PLACE] Date: [DATE]

[STUDENT NAME] | Signature of the Candidate

ACKNOWLEDGEMENT

I express my sincere gratitude to [PROJECT GUIDE NAME], [DESIGNATION], for guidance, review, and encouragement during the planning and preparation of this project report. I also thank the faculty and staff of the Department of Computer Applications at [COLLEGE NAME] for the academic support and resources made available during the course of the project.

I am grateful to my family, friends, and classmates for their encouragement and constructive feedback. I acknowledge the maintainers of the open-source frameworks and libraries used by Macrova, including Spring Boot, Spring Security, Spring Data MongoDB, Angular, and Chart.js, and the authors of the documentation cited in this report.

[STUDENT NAME]

ABSTRACT

Macrova is a web application for creating a goal-oriented diet plan and recording nutrition and body-weight progress. The codebase contains an Angular 19 client and a Java 21 / Spring Boot 3.4.4 REST backend. Spring Data MongoDB persists users, diet plans, daily tracking records, and AI food-suggestion records. JWT-based authentication protects API operations. The main user journey comprises account registration and login, diet-plan setup, daily weight and macro tracking, a progress dashboard, plan history, and meal suggestions generated through an OpenRouter-compatible large-language-model endpoint.

The planner estimates basal metabolic rate (BMR) with the Mifflin–St Jeor equation, applies an activity multiplier to estimate total daily energy expenditure (TDEE), and derives an intake target from the selected weight-change goal. Macro targets are represented as grams of protein, fat, and carbohydrate; daily targets are distributed over breakfast, lunch, dinner, and snack categories. The application supports tracking of consumed macro amounts and weight over a plan timeline.

ABSTRACT (CONTINUED)

This report describes the problem domain, requirements, architecture, domain model, algorithms, interface flow, implementation, verification approach, limitations, and future enhancements. The system is a software project and its calculations are estimates, not clinical advice. It does not integrate with medical records, wearable devices, food databases, or a registered dietitian workflow. The AI suggestion feature depends on external configuration and should be reviewed for accuracy, safety, and dietary suitability by users.

The repository currently includes one Spring application-context smoke test. This report therefore distinguishes implemented behavior from recommended test cases and does not claim a broad automated test suite or measured production performance. Configuration secrets, credentials, deployment values, candidate information, and institutional approval must be supplied and verified by the project owner before deployment or submission.

Keywords: Diet planner, nutrition tracking, Angular, Spring Boot, MongoDB, JWT, macronutrients, AI food suggestions.

TABLE OF CONTENTS

Title Page i

Bonafide Certificate ii

Declaration iii

Acknowledgement iv

Abstract v–vi

Table of Contents vii

List of Figures, Tables and Acronyms viii

Abbreviations and Terminology ix

Project Profile and Document Conventions x

CHAPTER 1: INTRODUCTION 1

1.1 Overview 1 | 1.2 Background 2 | 1.3 Problem Statement 3

1.4 Objectives 4 | 1.5 Scope 5 | 1.6 Assumptions and Organization 6 | 1.7 Profile 7

CHAPTER 2: LITERATURE REVIEW 8 | BMR 9 | Energy Balance 10 | Macronutrients 11

Personal Informatics 12 | Web Architecture 13 | Authentication and Data 14 | AI and Comparison 15–16

CHAPTER 3: SYSTEM ANALYSIS 17 | Existing and Proposed System 17–19 | Requirements 20–22 | Feasibility and Use Cases 23–25

CHAPTER 4: SYSTEM DESIGN 26 | Architecture, Flows and Use Cases 26–31 | Domain and API Design 32–38 | UI Design 39–40

CHAPTER 5: TECHNOLOGIES AND TOOLS 41

CHAPTER 6: IMPLEMENTATION 46 | Authentication and Plans 47–51 | Tracking and AI 52–58 | Algorithms 59–60

CHAPTER 7: TESTING AND RESULTS 61 | Existing Evidence and Proposed Test Plan 61–67 | Acceptance 68

CHAPTER 8: CONCLUSION AND FUTURE SCOPE 69–71

References 72–73

Appendix A: API and Data Dictionary 74–77

Appendix B: Calculations, Test Record and Operations 78–81

LIST OF FIGURES, TABLES AND ACRONYMS

Figures / textual design summaries: 4.1 Macrova high-level architecture; 4.2 request and token flow; 4.5 conceptual data relationships. Additional lifecycle, API, and calculation flows are documented in the relevant chapter sections.

Tables: 1.1 application profile; 1.2 project objectives; 2.1 comparison of solution categories; 3.1 functional requirements; 3.2 quality requirements; 3.3 feasibility; 4.1 domain entities; 4.2 API catalogue; 5.1 development stack; 7.1 test design; A.1 data dictionary; A.2 endpoint matrix.

Acronyms: AI—Artificial Intelligence; API—Application Programming Interface; BMR—Basal Metabolic Rate; BMI—Body Mass Index; CRUD—Create, Read, Update, Delete; DTO—Data Transfer Object; HTTP—Hypertext Transfer Protocol; JWT—JSON Web Token; LLM—Large Language Model; REST—Representational State Transfer; TDEE—Total Daily Energy Expenditure; UI—User Interface; UX—User Experience.

ABBREVIATIONS AND TERMINOLOGY

Angular: TypeScript-based client framework used for the single-page interface. BMR: an equation-based estimate of energy used at rest. TDEE: estimated daily energy expenditure calculated by multiplying BMR by an activity factor. Macronutrients: protein, carbohydrate, and fat, tracked in grams and converted to approximate kilocalories using conventional factors. Plan track: a dated record associated with a diet plan, including body weight, estimated energy values, macro totals, and meal allocations.

JWT: signed token used to convey an authenticated subject. Access token: short-lived token held by the client in memory in the current implementation. Refresh token: longer-lived token transported in an HttpOnly cookie. MongoDB collection: a set of document records. Repository: Spring Data interface used to query and persist a domain collection. Interceptor: Angular HTTP middleware for URL prefixing or authentication headers. OpenRouter: configured external API gateway used by the LLM service.

PROJECT PROFILE AND DOCUMENT CONVENTIONS

Product name: Macrova. Repository project: macrova-bca. Frontend package: macrova-ui. Backend Maven artifact: diet-planner. Backend application name: macrova. Primary feature areas: account access, diet-plan configuration, nutrition calculations, progress recording, dashboard/history, and AI-assisted meal suggestions.

This report uses “implemented” only for behavior visible in the checked-in source. “Proposed” or “recommended” denotes future work or verification that should not be inferred as currently complete. Endpoints are shown as backend paths; the frontend base-url interceptor prepends /api, and the development proxy configuration should be checked for the actual deployment environment. Values for student, guide, college, date, and approval remain placeholders because they are not present in the repository.

Version context observed in the codebase: Angular 19.2, Java 21, Spring Boot 3.4.4, MongoDB, and Chart.js 4.4.9. Verify versions against the build files if dependencies change.

CHAPTER 1 — INTRODUCTION

1.1 Overview of the Project

Macrova is a browser-based nutrition and weight-progress application. A user creates an account, enters personal measurements and a target, selects a goal and activity level, and receives an estimated daily energy and macro target. The application then supports daily progress entry and presents recent metrics and plan history. A separate meal discovery feature uses dietary and cuisine filters to request a small set of AI-generated suggestions.

The application is divided into an Angular single-page frontend and a Spring Boot REST API. The backend derives the current account from the authenticated security context rather than accepting the user identifier as a client-selected owner for core diet-plan operations. MongoDB collections store users, plans, tracking records, and food suggestions.

1.2 Background and Motivation

Nutrition planning involves more than choosing a target weight. A useful personal tracker needs an accessible way to convert basic body measurements and activity into an approximate energy target, translate that into daily macro goals, and compare intended intake with recorded intake over time.

Paper logs and generic spreadsheets can record numbers but often require users to calculate each target and organize meals manually. A web application can place those calculations, daily records, progress summaries, and historical plans in a single workflow. Macrova is motivated by that need for a compact, self-service tool rather than a clinical decision-support system.

The application’s approach favors transparent, rule-based calculations for baseline targets and an optional LLM integration for food ideas. These functions serve different purposes: calculations are deterministic code, while suggestions are external generated content that may vary and require user judgment.

1.3 Problem Statement

A person attempting to manage body weight may not have a consistent way to estimate a starting intake target, divide macronutrients into practical meal categories, and retain a dated record of changes in weight and consumption. Disconnected notes make trend review difficult; one-off calculators do not preserve a plan; and a general food search may not align with the user's selected meal, dietary pattern, cuisine, or available meal calorie budget.

The project addresses this software problem by combining plan setup, daily tracking, a dashboard, history, and filtered AI meal suggestions behind an authenticated web interface. It does not diagnose conditions, provide therapeutic diets, or verify that generated meal suggestions are nutritionally complete.

1.4 Objectives of the Project

The primary objective is to implement a usable web application that links diet-plan creation with routine progress recording. The codebase supports account registration and authentication; plan fields for goal, profile measurements, current and target weights, activity, and duration; energy and macro estimates; dated tracking; daily meal macro categories; recent progress metrics; plan history; and AI-generated food suggestions constrained by user-selected filters and a calorie budget.

Additional engineering objectives are to keep browser presentation separate from server business logic, protect API routes through Spring Security, scope plan access to the signed-in account in service queries, use typed DTOs and models across the UI, and keep external LLM configuration out of source code by reading it from environment-backed properties.

1.5 Scope and Boundaries

In scope are the browser UI, REST services, MongoDB persistence, JWT access/refresh flow, nutrition calculation utilities, the plan and track lifecycle, meal macro updates, and the OpenRouter-compatible suggestion call. The frontend routes include dashboard, progress, meals, plan history, plan form, login, signup, and related overview screens.

Out of scope are clinical validation, prescription or diagnosis features, payment, social networking, barcode scanning, automatic food logging, wearable integrations, offline synchronization, administrator workflows, and a professionally curated nutrient database. The application does not establish that an entered weight, activity level, food description, or AI output is accurate.

1.6 Assumptions, Constraints and Report Organization

Assumptions: users can access a modern browser and the deployed API; the backend has a reachable MongoDB database; and AI suggestions require valid OpenRouter model and API-key configuration. The environment properties require JWT_SECRET, OPENROUTER_MODEL, and OPENROUTER_API_KEY. A MongoDB URI is commented in the checked-in properties, so a deployment-specific connection mechanism must be provided.

Constraints include reliance on external services, unverified medical appropriateness, fixed supported profile enums, and limited existing automated tests. Chapter 2 reviews relevant methods and frameworks; Chapter 3 analyzes requirements; Chapter 4 specifies architecture and data; Chapters 5–6 describe tools and implementation; Chapter 7 defines verification and observed evidence; Chapter 8 concludes and proposes enhancements.

1.7 Application Profile

Table 1.1 — High-level profile

Product: Macrova | Application type: responsive web application | Client: Angular 19.2 / TypeScript | API: Java 21 / Spring Boot 3.4.4 | Persistence: Spring Data MongoDB | Authentication: JWT access token with refresh-token cookie | Charts: Chart.js dependency | AI provider interface: OpenRouter chat completions endpoint | Main domain: account, diet plan, plan track, food suggestions.

The profile describes repository dependencies and code structure, not a production deployment. No public endpoint, live database, cloud environment, or availability objective is configured in the supplied project tree.

CHAPTER 2 — LITERATURE REVIEW

2.1 Introduction

Macrova combines ideas from nutritional estimation, personal informatics, web application architecture, token-based authentication, document-oriented persistence, and generative AI integration. This review identifies the concepts relevant to the implementation and separates established methods from product claims. The project is an engineering implementation rather than a clinical study; no user trial or controlled health outcome evaluation is included in the repository.

2.2 Basal Metabolic Rate Estimation

The Mifflin–St Jeor equation is used by the NutrientsCalculator utility to estimate resting energy needs. For the supported male/female enum, the implementation uses BMR = 10 × weight(kg) + 6.25 × height(cm) – 5 × age(years) + S, where S is +5 for male and –161 for female. It is a population-derived estimate and not a direct measurement of an individual's metabolism.

Macrova multiplies the result by an activity coefficient to obtain a TDEE estimate. Both the underlying equation and selected activity coefficient affect the result; any imprecision in age, measurements, activity, or classification propagates into the daily target.

2.3 Energy Balance and Goal Adjustment

The system applies a goal intensity percentage to estimated TDEE for weight-loss plans and applies a moderated surplus for weight-gain plans. The code's goal percentages are 15, 20, and 25. The deficit helper subtracts the selected percentage from TDEE; the surplus helper adds the percentage minus ten points. Results are rounded by a helper to a 50-kcal increment or next 100-kcal value according to its remainder logic.

These are implementation rules, not guarantees of a particular rate of change. Real weight trends are affected by fluid variation, health conditions, adherence, and other factors. A safe production product should communicate uncertainty and encourage qualified guidance.

2.4 Macronutrient Accounting

Macrova stores daily targets and consumed amounts as protein, fat, and carbohydrate values. Its calorie accounting follows common approximations: 4 kcal/g for protein, 9 kcal/g for fat, and 4 kcal/g for carbohydrate. Target energy is allocated 30% to protein, 30% to fat, and 40% to carbohydrates, then converted to grams. The implementation computes the values independently and rounds to whole grams; rounding can mean the displayed macro energy total differs slightly from the rounded calorie target.

Meal allocations in DietPlanService split each macro total into breakfast 30%, lunch 30%, dinner 15%, and snack 25%. Those shares sum to 100%, but the names and proportions are defaults rather than individualized meal prescriptions.

2.5 Personal Informatics and Progress Tracking

Personal informatics systems help people collect, review, and reflect on data about themselves. Macrova's daily plan-track records combine weight with computed energy and macro targets, while the dashboard retrieves a recent date range and the history views support review across completed plans.

The usefulness of a trend depends on consistency, date interpretation, data quality, and feedback design. Macrova uses LocalDate and custom MongoDB conversions, and its metrics logic explicitly uses the Asia/Kolkata time zone for a one-week range. Testing should include daylight/date-boundary behavior, missing days, repeat submissions, and long inactive periods.

2.6 Web Architecture and Separation of Concerns

Angular organizes the browser application around routes, components, injectable services, DTOs, and HTTP interceptors. Spring Boot organizes the server around controllers, services, repositories, domain entities, configuration, security filters, and exception handling. The client submits data over HTTP; server services apply ownership checks and calculations; repositories translate domain operations to MongoDB queries.

Separating presentation and domain logic provides a clear contract boundary, but it does not automatically ensure validation consistency. Constraints should be enforced server-side even when the client also validates fields. Contract tests and OpenAPI documentation would strengthen the boundary.

2.7 Token Authentication and Document Databases

JWTs allow a stateless API to validate a signed subject and expiry. Macrova signs tokens using HMAC512 and reads the secret from JWT_SECRET. The login filter issues a refresh token in an HttpOnly cookie; refresh returns an access token; the Angular client keeps the access token in memory and requests a refresh when needed. This design reduces persistent browser storage of the access token but still requires strict production cookie, CORS, CSRF, expiry, and secret-rotation decisions.

MongoDB documents can naturally represent nested macro structures and track snapshots. Spring Data repositories provide query derivation and projection support. Document databases do not remove the need for indexes, ownership predicates, schema evolution, atomicity analysis, and data-retention policy.

2.8 Generative AI for Food Discovery

An LLM can produce readable meal ideas from natural-language constraints, but generated text is probabilistic and can contain incorrect nutrition values, missing allergens, or unsuitable instructions. Macrova's OpenRouterLLMService requests JSON containing name, calories, macronutrients, and description. AIFoodSuggestService builds a prompt from meal, food, cuisine, and a calorie budget retrieved from the plan service, then parses the returned JSON into application entities.

Reliable systems should constrain structured outputs, validate numeric ranges, make uncertainty visible, guard against prompt injection, apply timeouts and rate limits, and avoid presenting generated nutrition as verified. The current source does not show a food database cross-check or a user-facing clinician review workflow.

2.9 Comparative Study and Research Gap

Table 2.1 — Solution categories

Paper / spreadsheet: flexible but manual calculation and fragmented history. Standalone calorie calculator: quick estimate but limited longitudinal tracking. Generic food log: broad logging but may not bind entries to a goal-specific daily plan. Macrova: ties plan estimation, daily macro tracking, weight history, and filtered AI meal discovery together in one codebase.

The project gap is a focused, integrated workflow for personal plan setup and progress journaling. This comparison is qualitative and based on feature categories, not a benchmark against named commercial products. The remaining gap is validation: a nutrition-quality dataset, security review, user study, and comprehensive automated test suite are not supplied.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Existing System Analysis

A common baseline workflow uses a calculator for energy needs, a separate notes app or spreadsheet for weight, and general web search for meal ideas. The user must manually carry targets between tools, reconstruct trends, and decide whether suggestions fit meal-level goals. Macrova consolidates those operations while retaining a modular server/client design.

3.2 Limitations of the Existing Workflow

Manual tools can produce inconsistent units, rounding, and duplicate entries. A one-time calculator does not create a durable plan timeline. A spreadsheet may be difficult to use on mobile and does not inherently enforce account-level access. Search results generally do not know a user's current meal calorie budget. These observations motivate an integrated application but are not quantified survey findings.

Existing commercial nutrition products offer different data sources and feature sets. This report avoids claims of superiority because no formal comparison, user study, or performance benchmark was performed.

3.3 Proposed System

The proposed solution is the current Macrova application: Angular interface, Spring REST services, MongoDB persistence, JWT security, deterministic nutrient calculations, and optional LLM meal generation. Users enter plan information, the server calculates target values and initializes a tracking record, and subsequent requests update daily weight or meal macro intake.

The server is authoritative for plan ownership and calculation; the browser handles form presentation and route navigation. The AI integration is an external dependency, while the core diet-plan and tracking features are local backend functions.

3.4 Advantages and Expected Outcomes

The application centralizes related personal tracking tasks, gives each account a plan history, reuses daily calculations, and offers a chart-oriented recent view. Meal categories provide a manageable way to record intake without requiring every ingredient to be logged. Filtered suggestions create a practical bridge from calorie targets to meal ideas.

Expected outcomes are software usability and continuity of records, not clinical weight loss. Success should be evaluated through correct calculations, authorization isolation, reliable persistence, understandable interfaces, and feedback from representative users.

3.5 Functional Requirements — Account and Plans

FR-01: register an account with username, email, and password. FR-02: authenticate credentials and establish token-based access. FR-03: retrieve the current username for the interface. FR-04: create a plan with title, plan type, age, gender, height, current weight, target weight, goal intensity, duration, and activity. FR-05: retrieve the signed-in user’s active plan and individual plans. FR-06: return a TDEE estimate from submitted body metrics.

FR-07: validate plan targets and duration on the server. FR-08: create an initial tracking snapshot when a plan is created. The UI additionally performs field validation, but server-side checks remain necessary because clients can be bypassed.

3.6 Functional Requirements — Tracking and Suggestions

FR-09: record or revise a plan’s current-day weight. FR-10: retrieve the current day’s track and meal allocation. FR-11: update protein, fat, and carbohydrate consumed values for a meal category. FR-12: retrieve recent metrics and plan history/overview. FR-13: determine whether a plan duration has elapsed and deactivate an expired plan. FR-14: suggest foods from meal, food preference, and cuisine filters using the configured LLM.

FR-15: show authentication, validation, empty-plan, loading, and suggestion-error states through the browser interface. The application contains a toast component and route guard. Detailed usability behavior should be confirmed through manual acceptance testing.

3.7 Non-Functional Requirements

Security: sensitive endpoints require authentication; plan lookups should be scoped to the authenticated user; secrets are externalized. Reliability: database failures and AI-provider failures should return controlled errors. Usability: forms explain units and constraints, routes remain coherent, and progress is readable. Maintainability: feature packages, typed DTOs, services, and repositories separate responsibilities.

Performance: common operations should avoid unnecessary queries and use appropriate MongoDB indexes. Compatibility: modern browser support is expected from the Angular build. Accessibility: keyboard operation, labels, contrast, and screen-reader behavior should be verified; source presence alone does not demonstrate conformance. Privacy: collect only necessary profile data, define retention, and communicate external AI data handling.

3.8 Feasibility Study

Technical feasibility: the selected technologies are established and the repository contains both client and server code. Operational feasibility: a user can interact through standard browser routes, but the system requires MongoDB and, for meal suggestions, provider configuration. Economic feasibility: open-source frameworks reduce licensing barriers; hosting, database operation, and LLM calls may still incur cost.

Schedule feasibility depends on validation, testing, and deployment scope. Legal and ethical feasibility requires privacy review and careful positioning of health-related estimates. These are project-level assessments, not a formal institutional approval or cost calculation.

3.9 Use Cases and Acceptance Criteria

Actor: registered user. Main use cases: sign up, log in, create a plan, view dashboard, record weight, update meal macros, review history, request meal ideas, and log out. A guest can access registration and login; protected areas use AuthGuard.

Acceptance criteria should verify: invalid measurements are rejected; a plan is associated with its creator; unauthorized requests fail; plan history is limited to the owner; target equations match expected test vectors; consumed macro changes recalculate daily totals; a missing/expired plan leads to the appropriate no-active-plan state; and provider errors do not leave the suggestion screen indefinitely loading.

CHAPTER 4 — SYSTEM DESIGN

4.1 Design Overview

Macrova follows a client–API–persistence architecture. Angular components render forms and summaries, Angular services make HTTP requests, interceptors add the /api prefix and authorization header, Spring controllers expose REST resources, services enforce business rules, repositories query MongoDB, and document entities represent persistent state.

Design goals are clear module boundaries, owner-scoped data access, reusable calculations, explicit API models, and graceful separation between deterministic planning and external AI output.

4.2 High-Level Architecture

Figure 4.1 — Component flow (logical): Browser / Angular routes → Components → Angular services → HTTP interceptors → /api REST endpoint → Spring Security filters → Controllers → Domain services → Spring Data MongoDB repositories → MongoDB database. The food-suggestion path additionally calls LLMService → OpenRouter chat completions API.

The application is a modular monolith on the backend, not a microservice deployment. The frontend and backend may be developed and hosted separately, but the source does not prescribe a production topology.

4.3 Request and Authentication Flow

Figure 4.2 — Authentication flow: credentials POST to /authenticate → custom AuthenticationFilter reads username/password → CustomAuthenticationManager finds account and matches encoded password → success issues an HttpOnly refresh cookie → client calls /user/refresh when an access token is absent or expired → server validates refresh token and returns a short-lived access token → Angular TokenInterceptor adds Bearer token to protected calls.

SecurityConstants specifies 3,600,000 ms access and 86,400,000 ms refresh expiration. Production settings should use HTTPS secure cookies and an explicit allowed-origin policy. The current cookie builder sets secure(false), and CORS allows only http://localhost:4200; those values are development-oriented and must be reviewed for deployment.

4.4 Frontend Component Design

The Angular application uses standalone-style feature components and route declarations. App routes include /login, /signup, /dashboard, /progress, /meals, /history, /history/:id, /history/track/:id, /diet-plan-form, /no-active-plan, and /not-found. AuthGuard protects the main workflow routes, although the two history detail routes in the checked-in routing array do not declare canActivate.

Feature services isolate HTTP calls: AuthService handles identity/token actions; DietPlanService wraps plan and tracking endpoints; AIFoodSuggestService submits filter choices. Interceptors apply the API prefix and token behavior.

4.5 Backend Package Design

The backend root package is org.karthick.macrova. Major packages include user (controller, DTOs, entity, repository, service); dietplan (controller, DTOs, entity, enums, repository, service); aifoodsuggest; aichatservice; security; config; exception; shared models and mapper; and util calculations/constants.

Controllers adapt HTTP input/output. Services coordinate domain operations. Repositories declare MongoDB query methods. DTOs shape API contracts; entities map to collections. Security filters parse credentials/tokens and populate the SecurityContext. ApplicationExceptionHandler and filter-level handling translate error cases.

4.6 Use Case Specifications — Registration and Login

Registration accepts user details at POST /user/register, checks username and email uniqueness, encodes the password through Spring PasswordEncoder, and saves a User document. Login posts credentials to /authenticate. On success, the filter returns a refresh cookie. The client requests the username and uses the refresh endpoint to obtain an access token as needed.

The current registration controller receives a User entity directly rather than a narrowly scoped registration DTO, so API hardening should explicitly whitelist accepted fields and apply validation. Email normalization, password policy, verification, rate limiting, refresh-token revocation, and account recovery are not established in the current source.

4.7 Use Case Specifications — Diet Plan Lifecycle

The user enters plan characteristics through DietPlanFormComponent. Client checks require title, age 18–65, height 140–210 cm, weight 40–180 kg, target direction consistent with loss/gain, and duration 14–730 days. Backend validation checks a target BMI boundary and goal-duration interval. On creation, the service sets todayWeight from weight, associates authenticated user ID, activates the plan, persists it, and creates the first daily tracking record.

The implementation's updateDietPlan method delegates to createDietPlan; it should be reviewed before interpreting the route as an in-place update. Plan replacement semantics and multiple active plans require careful verification against repository behavior.

4.8 Domain Model and Relationships

Figure 4.5 — Conceptual relationship: User 1 → many DietPlan; DietPlan 1 → many DietPlanTrack; AIFoodSuggest documents are suggestion-shaped records. A plan stores profile inputs and active state. A track stores a date, weight, day count, TDEE, target/taken calorie values, macro values, meal-level values, and dietPlanId. User IDs and plan IDs are represented as strings.

MongoDB references are application-level identifiers rather than relational foreign keys. The source has no explicit cascade policy or schema migration framework. Deletion, anonymization, orphan cleanup, and data-retention behavior should be documented and implemented if required.

4.9 Data Dictionary — User and DietPlan

User collection: id (MongoDB _id, string), username (string), email (string), password (encoded credential string). The current User entity has no role, verification state, audit timestamps, or profile fields.

DietPlan collection: id; title; plan enum (WEIGHT_LOSS / WEIGHT_GAIN); age; gender enum; height in centimetres; initial weight and todayWeight in kilograms; goal enum (MILD / MODERATE / AGGRESSIVE); targetWeight in kilograms; duration in days; activity enum; userId; active boolean; createdAt LocalDate. Date conversion maps LocalDate to UTC-midnight Date and back.

4.10 Data Dictionary — DietPlanTrack and Suggestions

DietPlanTrack collection: id; weight; tdee; intake (Calories); protein, carbs, fat (Calories objects); mealKcal (MealKcal); dayCount; createdAt; dietPlanId. Calories/Macros models represent target and consumed quantities; MealKcal groups breakfast, lunch, dinner, and snack macro allocations.

AIFoodSuggest collection: id; name; calories; macronutrients; description. AIFoodSuggestService parses model JSON into these objects. The repository contains an addSuggestions persistence method, but the main suggestFood flow shown in source returns parsed suggestions and does not call addSuggestions. Avoid assuming suggestion history is stored unless another caller is introduced.

4.11 API Design — Authentication and User

POST /authenticate — authenticate JSON credentials; returns refresh cookie on success. POST /user/register — register user. POST /user/refresh — validate Refresh-Token cookie and return AccessTokenDTO. POST /user/logout — clear refresh cookie. GET /user/username — return authenticated username.

The Angular environment sets baseUrl to /api, so requests such as /user/username are transformed to /api/user/username. The proxy file should be checked for actual development forwarding. Production should define TLS, domain, cookie attributes, allowed origins, error schemas, and throttling.

4.12 API Design — Diet Plans (Read and Create)

GET /diet-plan/all — list the current account's plans. GET /diet-plan/history — history projection. GET /diet-plan — current active plan. GET /diet-plan/{dietPlanId} — plan by identifier with ownership check in the service. POST /diet-plan/tdee — calculate a TDEE response from body metrics. POST /diet-plan — create plan and initial track. PUT /diet-plan/update — invoke update service behavior.

The endpoint listing reflects controller mappings. Response shape and HTTP status details should be confirmed with integration tests or API documentation. A production API contract should include request validation, error examples, pagination where needed, and authorization requirements.

4.13 API Design — Tracking, Metrics, and AI

POST /diet-plan/add-weight/{dietPlanId} — update daily weight. GET /diet-plan/track/{dietPlanTrackId} — track by ID. GET /diet-plan/track/plan/{dietPlanId} — retrieve or create today's track. POST /diet-plan/{category}/{dietPlanId} — update meal macros for breakfast/lunch/dinner/snack. GET /diet-plan/meal-kcal/{id} — return meal allocations. GET /diet-plan/metrics — recent metrics. GET /diet-plan/active-plans — active-plan existence. GET /diet-plan/overview/{dietPlanId}, /status, and /past-plan — supporting views and status.

POST /ai-food-suggest — submit FilterDTO and return a list of suggestions. All protected endpoints are expected to carry a Bearer token.

4.14 Validation, Errors, and Security Design

Client-side validation improves feedback but cannot be trusted as a control. The service resolves user identity from UserSession and scopes plan reads by userId for key operations. JWT signatures are verified by JWTTokenService. Registration and refresh are explicitly permitted in SecurityConfig; other requests require authentication.

Exception types include bad request, entity not found, and unauthorized. The filter layer catches token verification and runtime errors. Error responses should be checked for consistent JSON shape, correct status mapping, non-leakage of internal details, and appropriate logs. The broad runtime exception handling currently maps runtime failures to a generic bad request at the filter boundary.

4.15 UI and Interaction Design

The application uses a dashboard-first protected flow, a diet-plan creation form, meal suggestion filters, history listing, and plan overview components. The route names indicate separate concerns, while shared header, toast, auth guard, DTOs, and services support consistency. The repository contains a design PNG labeled FitTrack rather than Macrova; it is not used here as evidence of the current Macrova runtime interface.

Recommended UI review criteria: clear units, accessible labels, loading and failure states, prevention of duplicate submissions, empty-plan guidance, mobile layout, readable chart axes, and explicit uncertainty around estimates. Screenshots should be captured from the running current application before final college submission.

CHAPTER 5 — TECHNOLOGIES AND TOOLS

5.1 Software Requirements

Backend build: Maven wrapper and Maven project. Java: release 21. Framework: Spring Boot 3.4.4. Data: MongoDB Spring Data. Security: Spring Security and Auth0 java-jwt 4.5.0. Mapping: ModelMapper 3.2.2. Utility: Lombok 1.18.38. Test dependency: Spring Boot starter test.

Frontend: Angular 19.2, TypeScript 5.7, RxJS 7.8, Zone.js 0.15, Chart.js 4.4.9, jwt-decode 4. Build and test commands are declared in frontend/package.json; Maven wrapper files are included in backend.

5.2 Hardware and Runtime Requirements

Development requires a computer capable of running a Java 21 JDK, Node.js/npm supported by Angular CLI, a browser, and a MongoDB server or managed MongoDB instance. Exact minimum CPU, RAM, storage, concurrent-user, and production capacity requirements have not been measured; they should be determined with realistic load testing.

At runtime, frontend hosting serves the built Angular assets; backend hosting runs the Spring Boot application and reaches MongoDB. The AI feature additionally requires outbound network access to OpenRouter and valid credentials/model selection.

5.3 Programming Languages and Frameworks

Java is used for controllers, services, domain entities, security, data access, and nutrition calculations. TypeScript is used for Angular components, guards, interceptors, DTOs, and services. HTML templates define component markup; CSS styles the UI. JSON is the API payload format and the model response structure expected by the suggestion service.

Spring MVC handles HTTP endpoints; Spring Security integrates authentication filters and authorization; Spring Data MongoDB provides persistence; Angular Router controls browser navigation; RxJS handles asynchronous HTTP flows; Chart.js supports charting in the declared dependency set.

5.4 Configuration and External Services

backend application.properties declares spring.application.name=macrova and reads jwt.secret, llm.openrouter.model, and llm.openrouter.api-key from environment variables. A sample MongoDB URI is commented out. Secrets must not be committed. Environment-specific Mongo connection, allowed origins, TLS termination, logging, and health checks should be established for each deployment profile.

OpenRouterLLMService posts to https://openrouter.ai/api/v1/chat/completions with Authorization: Bearer API key. The model is chosen from configuration. Timeout, retry, quota, and response-size policies should be explicitly configured before production.

5.5 Development Workflow and Version Control

The repository has backend and frontend directories plus exampleDocs containing the reference BCA report. A practical workflow is: create a feature branch; implement a focused frontend/backend change; run Maven tests and Angular build/tests; review API compatibility; inspect security-sensitive diffs; update documentation; and validate a clean deployment configuration.

The report is prepared from the checked-in repository and reference report format. Before academic submission, confirm the project title, author and guide fields, screenshots, institution-specific declarations, source-code version, citations, and actual execution evidence.

CHAPTER 6 — IMPLEMENTATION

6.1 Module Description

Macrova is implemented as cooperating frontend and backend feature packages. The current source includes registration/login, plan configuration, calculation, daily tracking, progress dashboard/history, and AI food suggestions. This chapter describes code behavior rather than claiming features absent from source.

6.1.1 User Registration and Credential Handling

UserController exposes POST /user/register. UserService checks whether username and email already exist, encodes the submitted password with PasswordEncoder, saves the User document, and converts it to UserDTO. The public User DTO avoids returning the password field through normal registration output.

Improvements recommended: accept a dedicated input DTO, validate email and password formats, normalize username/email consistently, add unique database indexes to eliminate concurrent duplicate-registration races, verify email if required, and test that errors never expose encoded or raw credentials.

6.1.2 Login, Refresh, and Logout

AuthenticationFilter parses the request body as User and submits username/password authentication. CustomAuthenticationManager retrieves a user by username and compares the password using PasswordEncoder. On success, a refresh token is signed and set as an HttpOnly cookie; the refresh endpoint verifies the cookie and returns a newly signed access token. Logout expires the cookie.

SecurityConstants defines the token lifetimes. The frontend AuthService stores the access token in a private in-memory field; TokenService checks expiry and isAuthenticated attempts a refresh when needed. Review cookie secure attributes, refresh-token rotation/revocation, logout behavior in client memory, and CSRF implications before production.

6.1.3 Diet Plan Form and Enums

DietPlanFormComponent offers weight loss/weight gain, male/female, mild/moderate/aggressive, and five activity levels. Client validation checks title and supported numeric ranges, then ensures the target direction is consistent with the plan. The service receives a DietPlanDTO and navigates to the dashboard after successful create/update.

Activity coefficients in Activity are 1.2, 1.375, 1.55, 1.725, and 1.9. Goal percentages are 15, 20, and 25. Supported profile and goal options are defined in backend enums and mirrored by Angular enums/dropdowns; API validation should reject unsupported values.

6.1.4 BMR and TDEE Calculation

NutrientsCalculator.findBMR implements the Mifflin–St Jeor structure. findTDEE multiplies the estimated BMR by the selected Activity value. The /diet-plan/tdee endpoint rounds the resulting TDEE with Math.round and returns the input profile together with the estimate.

For an illustrative example only, a 30-year-old male weighing 70 kg at 175 cm has estimated BMR = 10(70)+6.25(175)-5(30)+5 = 1,648.75 kcal/day. With the moderate activity factor 1.55, estimated TDEE is about 2,556 kcal/day before the endpoint rounding. This arithmetic is not individualized medical advice.

6.1.5 Goal Intake and Macro Targets

For weight loss, findDeficit subtracts the goal percentage from TDEE and rounds using roundCalories. For weight gain, findSurplus applies (goal percentage – 10)% to TDEE and rounds. findProtein assigns 30% of target calories to protein and divides by 4; findFat uses 30% divided by 9; findCarbs uses 40% divided by 4. Each output is rounded to whole grams.

Because calorie and gram values are rounded independently, exact energy equality is not guaranteed. Tests should capture the helper's boundary behavior, including the 50-kcal remainder cases, and verify positive values for all supported form ranges.

6.1.6 Plan Safety Checks and Creation

DietPlanService.validateDietPlan computes target BMI and rejects weight-loss targets below BMI 18 or weight-gain targets above BMI 25. It estimates minimum and maximum duration from weight difference and plan type, then rejects durations outside the range. On creation, service ownership is assigned from UserSession and a track is created for today.

The user-interface age, height, weight, target, and duration constraints are distinct from server BMI/duration checks. There is no evidence in the code that the formulas are clinically reviewed for all populations. Health-oriented messaging should describe these as software checks and defer special cases to qualified care.

6.1.7 Daily Tracking and Meal Allocation

A DietPlanTrack is initialized with current weight, date, day count, and plan ID. processDietPlan calculates TDEE and intake, derives macro targets, and assigns mealKcal. The meal split is 30% breakfast, 30% lunch, 15% dinner, and 25% snack.

GET /diet-plan/track/plan/{id} returns today's record or creates one if missing. POST /diet-plan/add-weight/{id} saves todayWeight, retrieves today's track, updates weight and calculated values, then saves. Repeated daily updates should preserve one record per date/plan; concurrent behavior should be verified and may need a unique compound index.

6.1.8 Macro Intake Updates

A client submits macro values to POST /diet-plan/{category}/{dietPlanId}. The controller validates category against breakfast, lunch, dinner, and snack. Service updates the selected meal's consumed protein, fat, and carbohydrate values, sums consumed quantities across all meals, calculates calories from the 4/9/4 factors, and persists the track.

The current category switch defaults to snack for a non-matching string, but the controller validates first. Server-side numeric range and negative-value checks should be added. The API should also verify that the named plan belongs to the authenticated user before updating its track.

6.1.9 Dashboard and Recent Metrics

DashboardComponent first checks duration status and navigates to no-active-plan when required. Otherwise it requests metrics and handles a not-found response by returning to the no-active-plan route. DietPlanService obtains a one-week date range using Asia/Kolkata, queries plan-track records, and builds labels and weights for the view.

The dashboard also includes a weight-track component and a pipe for limiting display. Good dashboard testing should cover a new plan with one point, missing days, timezone boundaries, exactly-at-duration and beyond-duration days, and a database query with no records.

6.1.10 Plan History and Overview

DietPlanRepository exposes projections for a user’s history sorted by creation date; DietPlanService supplies a plan overview backed by track records. Angular routes include a history list, plan detail overview, and tracking detail. A user can revisit earlier plan information and use prior plan context when preparing a new plan form.

The detail routes /history/:id and /history/track/:id do not declare AuthGuard in the current route file, unlike core routes. Server authorization is therefore essential, and client guard coverage should be reviewed. The repository query for plan-by-id includes userId; overview and track lookups should receive equivalent ownership checks.

6.1.11 AI Food Suggestion Request

The UI exposes meal filters (Breakfast, Lunch, Snack, Dinner), food filters (Vegetarian, Non-vegetarian, Low-carb), and cuisine filters (Indian, Chinese, Japanese, Thai). The selected values populate FilterDTO. AIFoodSuggestService obtains the current meal calorie budget, builds a prompt requesting five foods, invokes LLMService, and parses a JSON array into AIFoodSuggest models.

If no active plan is found, getCaloriesByMealCategory currently returns 500L after catching EntityNotFoundException. This fallback should be reviewed: an explicit no-plan response is more transparent than an arbitrary calorie budget. The user interface should handle every error status so loading state always clears.

6.1.12 External LLM Adapter and Output Parsing

OpenRouterLLMService sends the configured model, system instruction, user prompt, temperature 0.3, max_tokens –1, and stream false to the chat completions endpoint. It reads choices[0].message.content and returns that string. AIFoodSuggestService then deserializes the output using Jackson into the target list type.

Robust production handling should include explicit timeouts, bounded tokens, rate limiting, JSON schema validation, null/empty response checks, safe error translation, and deterministic tests using a mock LLMService. Generated nutrient figures should be labeled unverified unless cross-checked against a trusted data source.

6.1.13 Persistence and Date Conversion

MongoConfig names the database macrova and registers LocalDate↔Date converters. LocalDate values are stored at UTC start of day and read in UTC, which helps avoid local zone shifts in persistence. DietPlan and DietPlanTrack map to diet_plans and diet_plan_tracks; users map to users; suggestion documents map to ai_food_suggests.

Operational readiness should add indexes for userId/active/createdAt and dietPlanId/createdAt, test index creation, and define data migrations or versioned compatibility rules. MongoDB connection credentials should be supplied through secure environment configuration, not source control.

6.1.14 Error Handling and Observability

ApplicationExceptionHandler maps domain errors to HTTP responses. ExceptionHandlerFilter handles token verification and runtime failures across the filter chain. Frontend components display some validation errors through ToastComponent and distinguish selected API statuses.

Recommended improvements include structured error DTOs, correlation IDs, redacted logs, metrics for external LLM latency/failure, clear client fallback behavior, and alerting for database connectivity. Error messages must not contain secrets or unnecessarily reveal whether an account exists.

6.1.15 Core Algorithm Pseudocode

Algorithm A — Create plan: receive validated form; obtain authenticated user ID; calculate target BMI and duration bounds; reject invalid plan; initialize todayWeight; mark active; save plan; compute BMR/TDEE/intake/macros; distribute macros into meal shares; create initial dated track; return plan.

Algorithm B — Record meal intake: verify meal category; load today's plan track; replace consumed macro quantities for the chosen meal; sum consumed protein/carbohydrate/fat across all four meals; calculate calories = 4P + 4C + 9F; save and return updated track.

Algorithm C — Suggest meal: validate filters; obtain daily meal calorie allowance; form constrained prompt; call configured LLM; parse structured response; validate output fields and limits; return safe suggestions or controlled error. Steps marked validate are recommended safeguards beyond basic current parsing.

CHAPTER 7 — TESTING AND RESULTS

7.1 Testing Strategy

Testing should cover utility-level calculation correctness, service rules, controller mappings and error behavior, MongoDB query semantics, Spring Security boundaries, Angular component flows, and full-stack integration. Since this is a health-adjacent application, authorization isolation and calculation boundary tests deserve particular attention.

The checked-in backend test suite currently includes MacrovaApplicationTests with a single contextLoads test. No further unit, UI, performance, or security results are asserted in this report. The following cases are a proposed verification matrix and should be marked PASS only after execution.

7.2 Existing Automated Test Evidence

The repository contains backend/src/test/java/org/karthick/macrova/MacrovaApplicationTests.java, annotated with @SpringBootTest, with one test method contextLoads(). It checks whether the Spring application context starts in the configured test environment.

This test is a smoke test; it does not demonstrate functional correctness of registration, JWT handling, MongoDB persistence, calculations, authorization, tracking, or AI behavior. It may also require environment variables and database connectivity depending on application configuration. The frontend package declares an Angular test script but no test files were identified in the source inventory at report preparation.

7.3 Proposed Unit Test Cases

TC-01: Mifflin–St Jeor BMR for male and female examples. TC-02: activity multipliers produce expected TDEE. TC-03: deficit/surplus and calorie-rounding edge cases. TC-04: macro gram values follow 30/30/40 calorie shares. TC-05: meal shares total 100%. TC-06: 4/9/4 consumed-calorie conversion. TC-07: BMI and duration validation for valid and invalid targets.

TC-08: category validation accepts only four supported values. TC-09: create plan assigns authenticated user and creates a first track. TC-10: meal update aggregates across meals. Use fixed clock/date dependencies where possible to make boundary tests deterministic.

7.4 Proposed API and Persistence Tests

Register with unique credentials; reject duplicate username/email; ensure output excludes password. Authenticate valid and invalid credentials; test missing, malformed, expired, and tampered JWTs. Verify refresh cookie attributes and access-token response. Ensure logout expires the cookie.

Persist and reload LocalDate fields without date shifts. Query history by owner and sort by newest. Prove one account cannot read/update another account's plan or track. Verify missing-plan and missing-track responses. Test same-day track idempotency and concurrent weight updates. These tests should use an isolated Mongo test database or an appropriate test container; avoid silently relying on a developer's local database.

7.5 Proposed Angular Tests

AuthGuard should allow authenticated routes and redirect unauthenticated users appropriately. AuthService should handle success, refresh, logout, expiry, and refresh failure. Diet-plan form should reject out-of-range and contradictory values. Dashboard should handle metrics, no plan, and service failures. Meal suggestion should submit selected filters and clear loading on both success and every error path.

Component tests should verify labels, keyboard interactions, accessible validation messages, and empty/error states. Add end-to-end flows for signup through plan creation and daily tracking. Do not treat a successful frontend build as proof of runtime correctness.

7.6 Security Verification Plan

Verify protected routes reject absent or invalid bearer tokens; ensure token subject maps to a persisted user; test identifier tampering for plans and tracks; ensure password hashes are never serialized; check error responses and logs for secrets; and confirm CORS is restricted to intended production origins.

Inspect production cookie Secure, HttpOnly, SameSite, Path, and expiry attributes; test refresh/logout lifecycle and CSRF assumptions; validate secret strength and rotation; apply request size/time limits; throttle authentication and AI usage; scan dependencies; and assess prompt injection and unsafe generated content. These are recommendations, not reported completed assessments.

7.7 Results, Limitations, and Quality Metrics

Observed repository evidence: source modules for the stated core features and one Spring context smoke test. No measured throughput, latency, user acceptance score, accessibility score, reliability percentage, or clinical outcome is available. Do not invent such numbers for academic presentation.

Suggested metrics include calculation test coverage, API authorization test coverage, frontend route coverage, defect counts by severity, provider failure rate, median/p95 response time under a stated workload, task completion time in usability sessions, and accessibility audit findings. A final project review should record test environment, database version, build command, result, and date.

7.8 Acceptance Test Checklist

Before release, demonstrate: account registration and login; secure access to protected routes; creation of a valid weight-loss and weight-gain plan; invalid form rejection; computed values against independently checked examples; weight tracking across dates; meal macro updates reflected in totals; history and overview rendering; plan expiry behavior; AI suggestions with valid configuration; graceful provider failure; and access denial for another user's identifiers.

Capture actual UI screenshots and anonymized test records. Replace this proposed checklist with execution evidence, including environment, test data, expected result, observed result, and signed review where required.

CHAPTER 8 — CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

Macrova demonstrates an integrated software workflow for diet-plan estimation, daily nutrition recording, progress review, and filtered AI meal ideation. It combines an Angular browser client with a Spring Boot API and MongoDB data model. Its calculation utilities encode BMR/TDEE, goal adjustment, macro conversion, and meal allocation; its service layer creates and updates plan tracking records; and its security layer uses signed JWTs with a refresh-cookie flow.

The implementation is a practical BCA project foundation, not a medically validated nutrition product. Quality claims remain bounded by the current source and limited automated test inventory.

8.2 Contributions and Current Limitations

Contributions include a modular full-stack implementation, explicit plan and track models, owner-aware service queries for key plan operations, reusable nutrition utilities, account authentication, historical progress views, and an adapter boundary for LLM meal suggestions.

Limitations include no verified food composition dataset, no clinician workflow, limited test coverage, unmeasured system performance, external LLM variability, no documented data-retention policy, and development-oriented CORS/cookie settings. API validation and owner checks should be audited across every path, particularly detail/overview/track endpoints. Target and duration constraints require domain review.

8.3 Future Enhancements

Near-term: add comprehensive backend and Angular tests; document API with OpenAPI; validate DTOs; harden registration, token rotation, rate limiting, secure cookies, CORS, and error schemas; add MongoDB indexes; implement centralized logging and health checks; and validate all ownership checks.

Product direction: searchable verified food database, custom recipes, allergen and dietary restrictions, configurable meal splits, trend smoothing, export/deletion controls, multilingual and accessible interface, wearable integration with consent, and carefully bounded personalization. Any health recommendations should be developed with qualified nutrition professionals and evaluated for safety, fairness, and privacy.

REFERENCES

[1] M. D. Mifflin et al., “A new predictive equation for resting energy expenditure in healthy individuals,” American Journal of Clinical Nutrition, vol. 51, no. 2, pp. 241–247, 1990. doi:10.1093/ajcn/51.2.241.

[2] S. J. Cunningham, “Body composition as a determinant of energy expenditure: a synthetic review and a proposed general prediction equation,” American Journal of Clinical Nutrition, vol. 54, no. 6, pp. 963–969, 1991.

[3] U.S. Department of Agriculture, “Dietary Reference Intakes: Macronutrients,” National Academies Press, 2005. doi:10.17226/10490.

[4] World Health Organization, “Obesity: preventing and managing the global epidemic,” WHO Technical Report Series 894, 2000.

[5] Spring, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/ (accessed October 2026).

[6] Spring, “Spring Security Reference,” https://docs.spring.io/spring-security/reference/ (accessed October 2026).

REFERENCES (CONTINUED)

[7] Spring, “Spring Data MongoDB Reference Documentation,” https://docs.spring.io/spring-data/mongodb/reference/ (accessed October 2026).

[8] Angular, “Angular Documentation,” https://angular.dev/ (accessed October 2026).

[9] MongoDB, “MongoDB Manual,” https://www.mongodb.com/docs/manual/ (accessed October 2026).

[10] Auth0, “java-jwt: Java implementation of JSON Web Token,” https://github.com/auth0/java-jwt (accessed October 2026).

[11] OpenRouter, “API Reference: Chat Completions,” https://openrouter.ai/docs/api-reference/chat-completion (accessed October 2026).

[12] Chart.js, “Chart.js Documentation,” https://www.chartjs.org/docs/ (accessed October 2026).

[13] K. Beck et al., “Manifesto for Agile Software Development,” 2001, https://agilemanifesto.org/.

Framework documentation citations identify technologies, not a claim that every documented capability is implemented in Macrova.

APPENDIX A — API AND DATA DICTIONARY

A.1 Authentication endpoint matrix

Method and path | Purpose | Authentication

POST /authenticate | Credentials; establishes refresh cookie | Public

POST /user/register | Create account | Public

POST /user/refresh | Refresh access token from cookie | Refresh cookie

POST /user/logout | Expire refresh cookie | Authenticated in SecurityConfig as currently configured

GET /user/username | Current username | Authenticated

All client requests may include /api prefix through BaseUrlPrefixInterceptor. Confirm exact proxy and production routing before use.

APPENDIX A — DIET PLAN ENDPOINT MATRIX

GET /diet-plan/all; GET /diet-plan/history; GET /diet-plan; GET /diet-plan/{dietPlanId}; POST /diet-plan/tdee; POST /diet-plan; PUT /diet-plan/update. These routes support user-owned plan listing, history, active plan retrieval, plan detail, TDEE estimation, creation, and update behavior.

POST /diet-plan/add-weight/{dietPlanId}; GET /diet-plan/track/{dietPlanTrackId}; GET /diet-plan/track/plan/{dietPlanId}; POST /diet-plan/{category}/{dietPlanId}; GET /diet-plan/meal-kcal/{id}; GET /diet-plan/metrics; GET /diet-plan/active-plans; GET /diet-plan/overview/{dietPlanId}; GET /diet-plan/status; GET /diet-plan/past-plan. Verify per-route ownership predicates through tests.

APPENDIX A — FOOD-SUGGESTION API AND ENUMS

POST /ai-food-suggest accepts FilterDTO and returns a list of suggestion records containing name, calories, macronutrients, and description. Valid frontend categories currently include meals Breakfast/Lunch/Snack/Dinner; food filters Vegetarian/Non-vegetarian/Low-carb; cuisines Indian/Chinese/Japanese/Thai.

Plan enums: WEIGHT_LOSS, WEIGHT_GAIN. Gender: MALE, FEMALE. Goal: MILD=15, MODERATE=20, AGGRESSIVE=25. Activity: SEDENTARY=1.2, LIGHT=1.375, MODERATE=1.55, HARD=1.725, ATHLETE=1.9. Meal category constants are lowercase breakfast, lunch, dinner, snack.

APPENDIX A — INPUT CONSTRAINTS AND DATA UNITS

Current frontend plan form limits: age 18–65 years; height 140–210 cm; current weight 40–180 kg; duration 14–730 days; weight-loss target less than current weight and at least 40 kg; weight-gain target above current weight and at most 110 kg. Backend additionally checks target BMI and goal-specific duration limits.

Units: height centimeters; body mass kilograms; duration days; calories kilocalories/day; macro quantity grams/day; dates LocalDate. These values are code constraints, not a medical recommendation. Confirm server-side validation parity before release.

APPENDIX B — CALCULATION REFERENCE

BMR = 10W + 6.25H − 5A + S; S=+5 for male and −161 for female in the implementation. TDEE = BMR × activity multiplier. Weight-loss intake = TDEE − (TDEE × goal percent / 100), rounded by roundCalories. Weight-gain intake = rounded TDEE + TDEE × (goal percent − 10)/100, then roundCalories.

Protein grams = round(30% of target kcal / 4). Fat grams = round(30% / 9). Carbohydrate grams = round(40% / 4). Consumed kcal = 4P + 9F + 4C. Meal macro shares: 30%, 30%, 15%, 25%. The helper uses Math.round for percentages and a custom 50/100 kcal rounding rule; tests must reflect the exact implementation rather than an assumed conventional rounding formula.

APPENDIX B — PSEUDOCODE AND TEST RECORD TEMPLATE

Test record: ID | build/commit | environment | precondition | steps | expected output | actual output | result | tester/date. Example proposed test: submit a meal macro update with a negative protein value; expected future behavior should be rejection after input validation is implemented. Do not record it as passing before a real run.

Security scenario: User A creates plan A; User B attempts to GET, update weight, and update meal macros by substituting plan A’s identifier. Expected: access denied or not found without revealing data. Execute only against isolated non-production data and attach the observed HTTP response.

APPENDIX B — DEPLOYMENT AND OPERATIONS CHECKLIST

1. Install compatible JDK 21 and Node/npm. 2. Configure MongoDB URI securely. 3. Set a cryptographically strong JWT_SECRET, OPENROUTER_MODEL, and OPENROUTER_API_KEY in the backend environment. 4. Confirm backend CORS origins and cookie Secure/SameSite/domain settings for HTTPS. 5. Configure frontend base URL and API proxy/reverse-proxy routes. 6. Install frontend dependencies and run Angular build/tests. 7. Run backend Maven tests and package the application.

8. Start services in a non-production environment and verify login, protected API, plan creation, tracking, and AI-provider failure handling. 9. Add database indexes and backups. 10. Review logs for secret redaction and define retention. 11. Perform security and privacy review before production. Exact deployment commands and values are environment-specific and should never include real secrets in this report.